

# TRACE AI: A Secure, Cost-Aware Trust Layer for Reducing Hallucinations in LLM Agents and RAG Systems

Deep Patel

TRACE AI

Ongoing research and engineering work

July 2026

**Abstract**—Large Language Models (LLMs) and retrieval-augmented generation (RAG) systems are increasingly used for research, support, enterprise workflows, coding, compliance, and agentic automation. However, current LLM systems still suffer from hallucinations, weak citation grounding, high inference cost, unsafe tool use, stale memory, prompt injection, and poor observability. This paper presents TRACE AI — Trustworthy Reasoning, Attribution, Cost, and Evidence, an independent research engineering project that proposes a modular trust layer between users, LLMs, RAG systems, and AI agents.

TRACE AI is built around CAGE — Claim-Aware Grounding and Evaluation, a verification pipeline that decomposes generated answers into atomic factual claims, grounds each claim in evidence, validates citations, repairs unsupported statements, and computes trust metrics. TRACE extends CAGE with CostGuard for cost-aware model routing, semantic verified caching, prompt compression, adaptive retrieval, quota reservation, and cost-per-verified-answer tracking. It also includes memory safety, domain-scoped memory retrieval, TTL-based re-verification, prompt-injection screening, agent tool authorization, human-in-the-loop gates, and EvalOps dashboards.

This research is not presented as a completed published system. It is an ongoing research and engineering effort that combines ideas from hallucination studies, truthfulness benchmarks, factuality evaluation, RAG evaluation, citation-grounded QA, guardrails, model routing, semantic caching, adaptive retrieval, active RAG, and efficient LLM serving. The goal is to build an open-source production-style architecture for AI systems that can deliver better, safer, more grounded, and more cost-controlled answers.

---

**Index Terms**—Large Language Models, Hallucination, RAG, Claim Verification, Citation Grounding, Trust Layer, AI Agents, Cost Optimization, Semantic Cache, Memory, Guardrails, EvalOps, CAGE, TRACE AI

---

## I. INTRODUCTION

LLMs can generate fluent and confident answers, but fluency does not guarantee truth. In many real-world applications, the central issue is not only whether a model can answer, but whether the answer can be trusted. A model may hallucinate facts, cite sources that do not support the claim, over-answer when evidence is weak, use stale memory, leak sensitive data, or take unsafe actions through tools. These problems become more serious in agentic workflows where LLMs can search, retrieve, call APIs, write files, send messages, or modify external systems.

TRACE AI addresses this gap by proposing a trust layer that sits between LLM outputs and final user-visible answers. The system does not assume that the first answer generated by an LLM is correct. Instead, it treats the answer as a draft

that must pass through claim extraction, evidence grounding, citation validation, risk-aware verification, answer repair, final safety checks, and scoring.

The project is named:

**TRACE — Trustworthy Reasoning, Attribution, Cost, and Evidence**

TRACE focuses on trusted answers, citations, cost control, verification, memory, agent security, and evaluation.

The core verification component is named:

**CAGE — Claim-Aware Grounding and Evaluation**

CAGE checks each claim, grounds it in evidence, validates citations, and reduces hallucinations.

The research question behind TRACE AI is:

How can we design a modular AI trust layer that reduces hallucinations, grounds answers in evidence, controls cost, manages memory safely, and supports secure LLM agents?

## II. PROBLEM STATEMENT

Modern LLM and RAG systems face six major production problems:

1. **Hallucination:** The model generates unsupported, unverifiable, or false statements.
2. **Citation weakness:** The model attaches citations that may not actually support the claim.
3. **Cost explosion:** Strong models, reranking, long context, and repeated retrieval can make trusted generation expensive.
4. **Memory risk:** Stored memory can become stale, irrelevant, poisoned, or privacy-sensitive.
5. **Agent risk:** Tool-using agents can act on unsafe instructions, prompt injection, or insufficient authorization.
6. **Evaluation gap:** Teams lack dashboards that show faithfulness, claim support, citation accuracy, cache savings, safety failures, memory quality, and model-wise tradeoffs.

TRACE AI proposes a system architecture that addresses these problems together rather than treating them as separate patches.

## III. RELATED WORK AND RESEARCH FOUNDATION

This section summarizes the research papers used in TRACE AI. Each paragraph answers four questions: (1) what problem the paper solves, (2) what method or algorithm it introduces, (3) what can be used in TRACE/CAGE/CostGuard, and (4) what limitation remains.

*A. Ji et al. — Survey of Hallucination in Natural Language Generation*

Ji et al. solve the problem that hallucination research in natural language generation was scattered across summarization, dialogue, data-to-text generation, machine translation, visual-language generation, and large language models. The paper does not introduce one algorithm; instead, it provides a taxonomy of hallucination causes, evaluation methods, mitigation strategies, and task-specific challenges. TRACE uses this survey as the broad motivation for why hallucination must be treated as a system-level problem rather than a single-model problem. The limitation is that a survey organizes existing work but does not directly provide a deployable verification pipeline; TRACE converts the survey’s problem framing into CAGE modules such as claim verification, evidence quality gates, citation validation, and final answer repair. [1]

*B. Lin et al. — TruthfulQA*

TruthfulQA solves the problem that many LLM benchmarks reward plausible answers even when those answers repeat common human misconceptions. The benchmark introduces 817 questions across 38 categories designed to test whether models generate truthful answers instead of imitative falsehoods learned from web text. TRACE uses TruthfulQA as a model and system evaluation benchmark for measuring whether CAGE reduces confident false answers, especially in domains such as health, law, finance, and politics. The limitation is that TruthfulQA evaluates final answers but does not itself build a full retrieval, citation, memory, cost, or agent-security system. [2]

*C. Kalai et al. — Why Language Models Hallucinate*

Kalai et al. solve the conceptual problem of explaining why hallucinations persist even in strong language models. The paper argues that hallucinations arise because training and evaluation procedures often reward guessing rather than calibrated uncertainty or abstention. The paper does not introduce a deployment algorithm, but it gives a key design principle: systems should not force models to always answer when evidence is weak. TRACE uses this insight in CAGE’s abstention logic, uncertainty labels, risk floors, and final verification pass. The limitation is that changing benchmark incentives is a field-level solution, while production systems still need concrete runtime mechanisms such as retrieval, claim verification, and source authority checks. [3]

*D. Li et al. — HaluEval*

HaluEval solves the problem of evaluating whether LLMs can recognize hallucinated content. It introduces a large-scale hallucination evaluation benchmark using generated and human-annotated hallucinated samples, along with a sampling-and-filtering framework for creating test examples. TRACE uses HaluEval as a red-team and regression benchmark for testing whether CAGE, LLM judges, NLI checkers, and guardrails can detect unsupported or conflicting statements. The limitation is that benchmark recognition is different from production prevention; TRACE must not only detect hallucinations after

the fact but also repair or abstain before the final answer reaches the user. [4]

*E. Min et al. — FActScore*

FActScore solves the problem that long-form generations often mix true and false facts, making binary factuality labels too coarse. The paper introduces a fine-grained evaluation method that decomposes generations into atomic facts and computes the percentage supported by reliable knowledge sources. TRACE uses this directly as the design foundation for CAGE’s atomic claim extractor, supported-claim rate, unsupported-claim rate, and trust score. The limitation is that FActScore is primarily an evaluation metric, while TRACE turns the idea into a runtime pipeline that extracts claims, retrieves evidence, validates citations, repairs unsupported claims, and writes verified claims to memory. [5]

*F. Manakul et al. — SelfCheckGPT*

SelfCheckGPT solves the problem of hallucination detection when the model is black-box and external databases or token probabilities may not be available. It introduces a sampling-based method: generate multiple responses for the same prompt and detect hallucination by measuring inconsistency across samples. TRACE can use SelfCheckGPT as an optional verifier when source evidence is unavailable or when checking whether a model is internally consistent on a claim. The limitation is that consistency is not the same as truth: multiple samples may agree on a false claim, so TRACE should use SelfCheckGPT as a weak signal alongside evidence retrieval, NLI, citation validation, and source authority checks. [6]

*G. Menick et al. — GopherCite / Verified Quotes*

GopherCite solves the problem that factual answers are difficult to trust unless users can inspect supporting evidence. It introduces an open-book QA model trained with supervised learning and reinforcement learning from human preferences to generate answers supported by specific evidence quotes and to abstain when unsure. TRACE uses this as direct inspiration for citation-grounded answering, evidence snippets, abstention, and final answer traceability. The limitation is that cited evidence alone is not sufficient: a quote can be real but irrelevant, outdated, cherry-picked, or not actually support the claim. Therefore, CAGE validates citation support claim-by-claim rather than only attaching citations. [7]

*H. Es et al. — RAGAS*

RAGAS solves the problem that RAG pipelines are hard to evaluate because retrieval quality, answer faithfulness, context relevance, and generation quality are separate dimensions. The paper introduces a reference-free evaluation framework for RAG systems, including metrics for context and answer quality. TRACE uses RAGAS-style metrics in EvalOps: faithfulness, answer relevancy, context precision, context recall, and retrieval quality. The limitation is that RAGAS is an evaluation framework, not a full trust architecture; TRACE combines such metrics with CAGE claim verification, CostGuard cost tracking, memory quality checks, and security red-team tests. [8]

### *I. Saad-Falcon et al. — ARES*

ARES solves the problem that manual RAG evaluation is expensive and difficult to scale. It introduces an automated RAG evaluation system that creates synthetic data, fine-tunes lightweight judge models, and evaluates RAG systems along context relevance, answer faithfulness, and answer relevance, using a small set of human annotations with prediction-powered inference. TRACE uses ARES as inspiration for scalable EvalOps: lightweight judge models can continuously evaluate retrieval, answer faithfulness, and answer relevance across domain-specific golden datasets. The limitation is that automated judges can drift or become miscalibrated; TRACE therefore includes golden-set regression, human approval for policy changes, and model-wise dashboard monitoring. [9]

### *J. Dong et al. — Building Guardrails for LLMs*

Dong et al. solve the problem that LLM safety guardrails are often fragmented and insufficient for diverse real-world applications. The paper surveys open-source guardrail systems and argues for systematic, context-aware, socio-technical guardrail design with verification and testing. TRACE uses this as the design basis for SecureCore and CAGE guardrails: input filtering, output filtering, PII checks, prompt-injection detection, scope checks, abuse/cost-DoS checks, tool authorization, and human review gates. The limitation is that guardrails can reduce risk but cannot guarantee truth or safety alone; TRACE combines guardrails with evidence grounding, domain policy, claim verification, and EvalOps. [10]

### *K. He et al. — DeBERTaV3*

DeBERTaV3 solves the problem of improving efficient pre-trained language representations for NLU tasks. It introduces replaced token detection with gradient-disentangled embedding sharing, improving training efficiency and downstream performance compared with earlier DeBERTa/ELECTRA-style approaches. TRACE can use DeBERTaV3-style models as lightweight classifiers or NLI backbones for claim verification, entailment checking, prompt-injection classification, domain classification, or reranking support. The limitation is that DeBERTaV3 is not a complete hallucination-reduction system; it provides a strong discriminative model component, but TRACE must integrate it into a broader retrieval, verification, repair, and scoring pipeline. [11]

### *L. FrugalGPT — Cost-Aware LLM Cascades*

FrugalGPT solves the production problem that using the strongest LLM for every query is expensive, wasteful, and sometimes not even the most accurate choice. The paper introduces an LLM cascade: send a query first to a cheaper model, estimate answer reliability, and escalate to stronger models only when needed under a user-defined budget. TRACE uses this as the design foundation for CostGuard’s escalation controller, especially after failed verification or low-confidence answers. The limitation is that FrugalGPT needs labeled or task-specific examples to tune the cascade, and it optimizes cost/accuracy more than safety, privacy, domain policy, tenant isolation, and calibrated abstention. [12]

### *M. RouteLLM — Preference-Based LLM Routing*

RouteLLM solves the problem of selecting between expensive strong LLMs and cheaper weaker LLMs before inference. The paper introduces routers trained on human preference data to predict whether a strong model is likely to outperform a weak model on a given query, then route based on a threshold. TRACE uses RouteLLM as the basis for CostGuard’s pre-generation model router: route easy or low-risk tasks to cheaper models and hard, high-risk, or low-confidence tasks to stronger models. The limitation is that the paper mainly focuses on strong-versus-weak routing and depends on training data matching the production distribution; TRACE must add domain risk, safety policy, memory freshness, and verification outcomes to routing. [13]

### *N. GPT Semantic Cache*

GPT Semantic Cache solves the problem of repeated LLM API calls for semantically similar user questions, which increases cost and latency. The paper introduces a semantic embedding cache using query embeddings, Redis, approximate nearest-neighbor search, cosine similarity, and cached responses when similarity is above a threshold. TRACE uses this as the basis for the Semantic Verified Cache, but adds tenant isolation, policy version, model fingerprinting, document version, freshness/TTL, verification score, and PII status before allowing a cache hit. The limitation is that semantic similarity does not guarantee answer equivalence; a similar query can require a different answer due to time, domain, policy, user context, or risk level. [14]

### *O. Adaptive-RAG*

Adaptive-RAG solves the problem that traditional RAG systems use the same retrieval strategy for every question even when some questions need no retrieval, some need single-step retrieval, and others need multi-step retrieval. The paper introduces a complexity classifier that selects among no-retrieval, single-step retrieval, and iterative retrieval strategies based on question complexity. TRACE uses this as CostGuard’s adaptive retrieval controller: simple stable questions avoid unnecessary retrieval, medium factual questions use normal RAG, and high-risk or complex questions use deeper retrieval, reranking, and verification. The limitation is that the classifier can misclassify query complexity and the paper does not directly address tenant isolation, PII, policy scope, freshness, or high-stakes domain rules. [15]

### *P. FLARE / Active RAG*

FLARE solves the problem that normal RAG retrieves only once before generation, which is weak for long-form answers that require evidence at different points in the answer. It introduces Forward-Looking Active Retrieval-Augmented Generation, where the model predicts upcoming content, uses low-confidence tokens as a signal to retrieve additional evidence, and regenerates the sentence with retrieved documents. TRACE uses FLARE as an active evidence controller inside CAGE: unsupported, contradicted, uncertain, or weakly cited claims trigger targeted retrieval rather than expanding all context upfront. The limitation is that active retrieval can create cost

and latency loops, so TRACE adds retry caps, quota checks, and forced abstention when evidence cannot be found. [16]

#### Q. vLLM / PagedAttention

vLLM/PagedAttention solves the LLM serving problem that high-throughput inference is limited by inefficient GPU KV-cache memory management. The paper introduces PagedAttention, inspired by operating-system virtual memory, splitting KV cache into fixed-size blocks and enabling near-zero waste plus sharing across requests. TRACE uses this in CostGuard’s serving optimizer when self-hosting open-source models for routing, verification, summarization, or generation. The limitation is that vLLM reduces infrastructure cost and improves throughput but does not itself reduce hallucination, validate citations, detect prompt injection, enforce domain policy, or manage memory safety. [17]

### IV. TRACE AI SYSTEM OVERVIEW

TRACE AI is designed as a modular architecture rather than a single monolithic pipeline. The uploaded TRACE architecture divides the full v4 flow into independent modules: security entry, input guardrails, classification/domain/risk, memory layer, CostGuard quota, semantic cache, execution modes, retrieval/ingestion, claim verification/repair, output safety/final verification, scoring, agent orchestration, EvalOps calibration, feedback/final output, and TTL reverification. This modular structure allows each part of the system to be tested, improved, and deployed independently.

The high-level flow is:

```
User input
v
SecureCore
v
Input guardrails
v
Domain/risk classification
v
Domain-scoped memory read
v
CostGuard planner + quota reservation
v
Semantic verified cache
v
Fast / Light / Full / Critical / Agentic route
v
Retrieval + ingestion + evidence pack
v
Claim extraction + verification + repair
v
Output safety + final verification
v
Scoring
v
Cache writeback + memory writeback
v
EvalOps + dashboard + feedback loop
```

The core idea is that TRACE does not trust generation alone. The LLM draft is treated as an unverified intermediate artifact until it passes evidence and safety checks.

### V. CAGE: CLAIM-AWARE GROUNDING AND EVALUATION

CAGE is the central trust engine in TRACE AI.

CAGE means:

#### Claim-Aware Grounding and Evaluation

CAGE performs the following steps:

1. Generate or receive a draft answer.

2. Split the answer into atomic factual claims.
3. Detect important facts such as dates, numbers, names, rankings, and table values.
4. Retrieve evidence for each claim.
5. Check whether evidence supports, contradicts, or fails to verify the claim.
6. Validate whether citations actually support the associated claim.
7. Apply source authority and freshness checks.
8. Repair unsupported or contradicted statements.
9. Add uncertainty or abstain when evidence is weak.
10. Compute trust metrics.

CAGE is inspired by FActScore’s atomic factual decomposition, GopherCite’s evidence-supported answers, SelfCheck-GPT’s consistency checking, and NLI-based verification using models such as DeBERTaV3.

### VI. COSTGUARD: COST-AWARE VERIFIED GENERATION

Trusted answers are usually more expensive than normal answers because verification requires retrieval, reranking, citation checking, and sometimes additional LLM calls. TRACE introduces CostGuard to make trusted generation affordable.

CostGuard includes:

- model routing
- LLM cascade escalation
- semantic verified cache
- adaptive retrieval
- prompt compression
- reasoning budget
- quota reservation
- async cost ledgering
- cost-per-verified-answer tracking

FrugalGPT motivates the cascade design. RouteLLM motivates pre-generation routing. GPT Semantic Cache motivates cache-based cost reduction. Adaptive-RAG motivates retrieval depth selection. FLARE motivates active targeted retrieval. vLLM/PagedAttention motivates serving optimization.

The main metric is:

$$\text{cost\_per\_verified\_answer} = \frac{\text{total\_request\_cost}}{\text{number\_of\_final\_supported\_claims}}$$

This metric is more useful than raw cost because a cheap unsupported answer is not actually valuable.

### VII. MEMORY AND TTL-BASED REVERIFICATION

TRACE includes memory, but memory is treated as evidence only after validation. Memory can help personalize answers and carry project context, but it can also become stale or poisoned.

Memory types include:

- user profile memory
- project memory
- verified claim memory
- episodic conversation memory
- Reflexion-style lessons
- tool history memory

Before memory is used, TRACE applies:

1. tenant and user scoping

2. domain filtering
3. freshness screening
4. memory injection screening
5. TTL checks
6. re-verification when needed

If a verified claim memory expires, a background TTL monitor rechecks it against fresh evidence. If the claim is no longer supported, the memory entry is invalidated and related semantic cache entries are purged. This prevents old verified claims from becoming future hallucination sources.

## VIII. AGENTGUARD: SECURE MULTI-STEP AGENT WORKFLOWS

TRACE supports agentic tasks but does not allow agents to bypass guardrails. Agentic tasks first pass through security, safety, PII checks, prompt-injection checks, domain classification, risk floors, memory policy, and CostGuard quota reservation.

AgentGuard includes:

- planner agent
- retrieval agent
- verifier agent
- repair agent
- security agent
- human review agent
- tool execution gate

Every tool call must pass:

1. re-authorization
2. tenant policy check
3. domain/risk policy check
4. sandbox execution
5. tool-output injection screening
6. observation returned to planner

High-impact tools under critical risk require human approval before execution. Examples include filing forms, sending external messages, deleting records, making payments, or modifying user data.

The agent loop has:

- max steps
- max cost
- max tool calls
- max wall-clock time
- budget exhaustion fallback

This prevents infinite agent loops and uncontrolled tool use.

## IX. DOMAIN POLICY ENGINE

TRACE is designed for multiple domains where evidence standards differ. A legal answer, medical answer, financial answer, immigration answer, code answer, and enterprise-support answer should not use the same verification rules.

The Domain Policy Engine controls:

- required source authority
- freshness requirement
- disclaimer requirement
- allowed answer style
- abstention threshold
- human review requirement
- tool permission level

- citation standard

Example domain rules:

```
Medical diagnosis / treatment -> Critical risk, strong
abstention, authoritative sources, disclaimer.
Legal filing advice -> Critical risk, jurisdiction check,
human review option.
Finance action advice -> High/Critical risk, freshness and
numeric verification required.
Immigration status/form filing -> Critical risk, official
government source preferred.
Research paper summary -> High trust required, paper-level
evidence and method/result extraction.
General education -> Medium or low risk depending on
factuality.
```

Policy conflicts are handled by a priority resolver. In practice, priority should consider safety severity, law, privacy, domain risk, organization policy, and user preference rather than relying only on a static global order.

## X. EVALOPS DASHBOARD

TRACE uses EvalOps to continuously measure trust, cost, security, memory, and agent behavior.

Important dashboard metrics include:

### A. CAGE metrics

- trust score
- supported claim rate
- unsupported claim rate
- contradiction rate
- citation accuracy
- source authority score
- freshness score
- abstention precision and recall

### B. RAG metrics

- context precision
- context recall
- answer relevancy
- faithfulness
- reranker lift
- retrieval hit rate
- evidence quality score

### C. CostGuard metrics

- cost per answer
- cost per verified answer
- tokens per answer
- cache hit rate
- verified cache hit rate
- model routing savings
- prompt compression savings
- latency p50/p95

### D. AgentGuard metrics

- task success rate
- tool call success rate
- unsafe tool attempt rate
- loop rate
- human approval rate
- policy violation rate

### E. Memory metrics

- memory precision
- memory recall
- stale memory rate
- memory conflict rate
- memory injection detection count
- TTL invalidation count

### F. Security metrics

- prompt injection attack success rate
- jailbreak success rate
- PII leakage rate
- tenant leakage rate
- tool permission bypass rate
- data exfiltration rate

EvalOps uses golden datasets, red-team tests, RAGAS-style metrics, ARES-style judges, HaluEval/TruthfulQA-style hallucination tests, and model-wise comparisons.

## XI. PROPOSED MVP

The full TRACE architecture is large, so the MVP should focus on the smallest version that proves value.

### A. MVP features

1. Input guardrails
2. Domain/risk classification
3. Hybrid retrieval over uploaded documents and web/document sources
4. Evidence pack builder
5. Draft answer generator
6. Atomic claim extraction
7. Citation support check
8. Claim labeler: supported / unsupported / contradicted / uncertain
9. Answer repair
10. Trust score
11. Cost ledger
12. Semantic verified cache
13. Basic EvalOps dashboard

### B. MVP experiments

The MVP should compare:

```
Baseline LLM answer
vs.
RAG answer
vs.
TRACE/CAGE verified answer
```

Measurements:

- supported claim rate
- citation accuracy
- hallucination rate
- cost per answer
- cost per verified answer
- latency
- cache savings
- user correction rate

## XII. EVALUATION PLAN

TRACE should be evaluated in three stages.

### A. Stage 1: Offline benchmark evaluation

Use public benchmarks:

- TruthfulQA for truthfulness
- HaluEval for hallucination recognition
- FactScore-style atomic claim support
- RAGAS for RAG faithfulness and context quality
- ARES-style automated RAG judging

### B. Stage 2: Domain-specific golden sets

Create internal golden datasets for the nine target domains. Each dataset should include:

- factual questions
- high-risk questions
- stale-current questions
- adversarial prompt-injection examples
- citation mismatch examples
- memory conflict examples
- ambiguous queries
- tool-use scenarios

### C. Stage 3: Production-style monitoring

Track live usage through dashboards:

- failed verification cases
- cache hit/miss reasons
- model route decisions
- abstentions
- human-review events
- user edits
- stale memory invalidations
- cost overruns

## XIII. LIMITATIONS

TRACE AI is still an ongoing research engineering project. The main limitations are:

1. **Implementation complexity:** The full architecture contains many modules and should be built in stages.
2. **Verifier reliability:** NLI models and LLM judges can be wrong.
3. **Citation ambiguity:** A source may partially support a claim but not fully support all details.
4. **Latency cost:** Claim-level verification and retrieval can slow down answers.
5. **Domain policy difficulty:** Legal, medical, finance, and immigration domains require careful policies and possibly human review.
6. **Memory governance:** User memory must be editable, deletable, exportable, and privacy-preserving.
7. **Eval judge drift:** Automated judges must be calibrated against human-labeled golden data.
8. **Cache safety:** Semantic cache reuse must be conservative for high-risk or time-sensitive answers.
9. **Agent risk:** Tool-using agents require strong permissions, sandboxing, and approval gates.
10. **Not a guarantee of truth:** TRACE reduces hallucination risk but cannot guarantee perfect correctness.

#### XIV. FUTURE WORK

Future work includes:

- implementing the MVP pipeline
- building benchmark datasets for selected domains
- adding model-wise leaderboard dashboards
- adding DeBERTaV3/NLI-based claim verification
- adding LLM judge calibration
- testing FrugalGPT-style cascades
- implementing RouteLLM-style routing
- adding semantic verified cache levels
- integrating TTL memory re-verification
- adding prompt-injection red-team tests
- preparing a formal paper submission after experiments

#### XV. CONCLUSION

TRACE AI proposes a secure, cost-aware trust layer for LLM agents and RAG systems. The core idea is simple: do not trust the first LLM answer. Instead, decompose answers into claims, ground those claims in evidence, validate citations, repair unsupported statements, control cost, manage memory safely, and evaluate the system continuously.

The project combines research ideas from hallucination surveys, truthfulness benchmarks, factuality metrics, citation-grounded QA, RAG evaluation, guardrail design, cost-aware model routing, semantic caching, adaptive retrieval, active RAG, and efficient LLM serving. CAGE provides the verification core; CostGuard makes verification affordable; AgentGuard makes tool use safer; memory and TTL re-verification prevent stale context from becoming future hallucinations; EvalOps keeps the system measurable.

This research is still continuing. The next step is to implement the MVP, run experiments, measure hallucination reduction, evaluate citation accuracy, compare model cost-quality tradeoffs, and refine TRACE AI into a practical open-source trust layer for production AI workflows.

#### REFERENCES

- [1] Ziwei Ji et al. *Survey of Hallucination in Natural Language Generation*. arXiv:2202.03629. <https://arxiv.org/abs/2202.03629>
- [2] Stephanie Lin, Jacob Hilton, Owain Evans. *TruthfulQA: Measuring How Models Mimic Human Falsehoods*. arXiv:2109.07958. <https://arxiv.org/abs/2109.07958>
- [3] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, Edwin Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664. <https://arxiv.org/abs/2509.04664>
- [4] Junyi Li et al. *HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models*. arXiv:2305.11747. <https://arxiv.org/abs/2305.11747>
- [5] Sewon Min et al. *FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. arXiv:2305.14251. <https://arxiv.org/abs/2305.14251>
- [6] Potsawee Manakul, Adian Liusie, Mark J. F. Gales. *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models*. arXiv:2303.08896. <https://arxiv.org/abs/2303.08896>
- [7] Jacob Menick et al. *Teaching Language Models to Support Answers with Verified Quotes*. arXiv:2203.11147. <https://arxiv.org/abs/2203.11147>
- [8] Shahul Es et al. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217. <https://arxiv.org/abs/2309.15217>
- [9] Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia. *ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems*. arXiv:2311.09476. <https://arxiv.org/abs/2311.09476>
- [10] Yi Dong et al. *Building Guardrails for Large Language Models*. arXiv:2402.01822. <https://arxiv.org/abs/2402.01822>
- [11] Pengcheng He, Jianfeng Gao, Weizhu Chen. *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing*. arXiv:2111.09543. <https://arxiv.org/abs/2111.09543>
- [12] Lingjiao Chen, Matei Zaharia, James Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. arXiv:2305.05176. <https://arxiv.org/abs/2305.05176>
- [13] Isaac Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data*. arXiv:2406.18665. <https://arxiv.org/abs/2406.18665>
- [14] Sajal Regmi, Chetan Phakami Pun. *GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching*. arXiv:2411.05276. <https://arxiv.org/abs/2411.05276>
- [15] Soyeong Jeong et al. *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity*. arXiv:2403.14403. <https://arxiv.org/abs/2403.14403>
- [16] Zhengbao Jiang et al. *Active Retrieval Augmented Generation*. arXiv:2305.06983. <https://arxiv.org/abs/2305.06983>
- [17] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. arXiv:2309.06180. <https://arxiv.org/abs/2309.06180>